CUỘC THI THIẾT KẾ VI MẠCH CHO ĐÔ THỊ THÔNG MINH LẦN 3 · BẢNG SINH VIÊN · BÁO CÁO BÁN KẾT

THIẾT KẾ VI MẠCH ĐIỀU KHIỂN BAY SỐ (`pid_ctrl`) CHO UAV QUADCOPTER TRONG ĐÔ THỊ THÔNG MINH

Từ Thuật toán Bay Thực tế trên STM32/ESP32 & Trạm Điều phối AI (LLM) đến Lõi IP/ASIC Fixed-Point Không Phép Chia

1. Tầng AI / LLM & GCS

- Trợ lý giọng nói tiếng Việt **Gemini Live** ra lệnh tác chiến đô thị.
- Buồng lái mô phỏng 3D **Three.js** + liên kết kiểm thử `$SIMTEL`.

2. Tầng PCB & Tay Cầm

- Bo mạch **Altium Custom PCB** tự thiết kế hoàn chỉnh.
- Màn hình PFD **ILI9341** + OLED, giao thức chung `RF_Protocol.h`.

3. Tầng UAV Quad Thực Tế

- Khung Carbon **Inventor 3D**, IMU **BNO085** (400 Hz) + GPS.
- Baseline MCU: **STM32F103C8** chạy `Flight.cpp` đã bay thực tế.

4. Tầng Vi Mạch Số (Trọng Tâm)

- RTL Verilog `pid_ctrl.v`: PID 3 trục + Mixer X-frame.
- **1 bộ nhân 14×29** dùng chung, trễ **20 clock (2 µs)**, **PASS 16.000/16.000**.

Mở đầu bằng: dự án đang ở đâu trên flow

Digital (pid_ctrl IP)	**Spec & architecture** 400Hz IMU, Python Q.10, 1-Multiplier FSM	**RTL** pid_ctrl.v (263 dòng) vlog -lint 0 warning	**Verification** Bit-exact PASS 16.000 / 16.000 mẫu	**Synthesis & STA** Cadence Genus Cấu hình A & B @ 10MHz	**P&R, DRC/LVS** Cadence Innovus & Pegasus Sign-off	**GDS / FPGA** Xuất GDSII + Nhúng FPGA HIL trên Drone
System (UAV & PCB)	**UAV & MCU Baseline** STM32F103 Flight.cpp & Khung Carbon 3D	**Schematic & PCB** Altium Custom PCB Tay cầm & Mạch bay	**3D HIL Simulator** GCS Three.js + $SIMTEL Serial Link	**MCU vs. ASIC Benchmark** Đo đạc Latency & Power STM32 vs. pid_ctrl	**SPI/UART Wrapper** Đóng gói giao tiếp 8-pin nối thẳng ESP32	**Closed-Loop Demo** LLM Voice ➔ Tay cầm ➔ Chip PID ➔ Drone

Đã xong (Hoàn thành 100% & có script tự động kiểm chứng) **Đang làm (Trọng tâm báo cáo Bán kết trên Cadence)** **Kế hoạch đến Chung kết**

Vì sao mở đầu bằng Flow: Mọi con số sau đó được hội đồng hiểu đúng mức tin cậy — nhóm tách bạch minh bạch giữa **kết quả kiểm thử RTL bit-exact (16.000 mẫu)**, **số liệu Post-Synthesis (Cadence Genus)** tại Bán kết và **số liệu Post-Layout (Cadence Innovus)** ở Chung kết.

Vì sao UAV Đô thị cần "Edge LLM + Vi mạch PID chuyên dụng"?

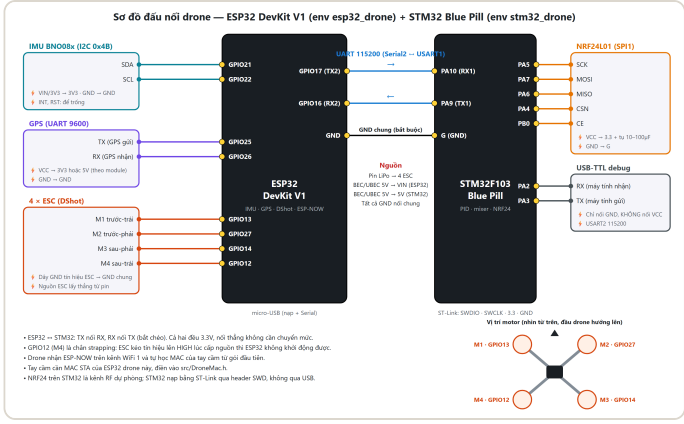
1. Bài toán thực tế trên Drone Quadcopter của nhóm

- Trong đô thị thông minh, UAV thực hiện tuần tra, giao hàng, cứu hộ cần xử lý đồng thời **2 tầng tác vụ trái ngược nhau**:
- **Tầng Nhận thức & Giao tiếp (High-Level Cognitive):** Nhận lệnh giọng nói tiếng Việt qua **LLM (Gemini Live)**, quản lý GPS, Geofence, RTH và truyền Telemetry Wi-Fi/ESP-NOW. Tác vụ này chạy trên `ESP32 / ESP32-S3` nhưng có độ trễ biến thiên (jitter vài ms đến chục ms).
- **Tầng Điều khiển Tư thế (Hard Real-Time 400 Hz):** Đọc góc nghiêng từ IMU `BNO085` mỗi **2.5 ms**, tính PID 3 trục (Roll, Pitch, Yaw) và trộn xung 4 động cơ Brushless. Nếu trễ nhịp hoặc treo task, drone lập tức lật cánh!

2. Hạn chế của giải pháp MCU truyền thống (STM32F103C8)

- Hiện tại nhóm phải dùng thêm 1 chip **STM32F103C8 (72 MHz)** chỉ để chạy hàm `Flight_Update()` (`src/Flight/Flight.cpp`) qua FreeRTOS.
- Cortex-M3 **không có FPU phần cứng**: mỗi phép nhân/chia `float32` phải giả lập bằng phần mềm, tốn ~1.300 chu kỳ lệnh (~18.5 µs) và tiêu thụ **~118.8 mW**.
- **Giải pháp của nhóm:** Chuyển toàn bộ lõi `UpdatePid() + Mixer` thành **vi mạch số chuyên dụng** `pid_ctrl` — không CPU, không phần mềm, tính xong cả 3 trục trong đúng **20 nhịp clock (2 µs @ 10 MHz)** với công suất dưới **1 mW**.

Kiến trúc Phần cứng Nguyên mẫu đã Chế tạo



Hình thực tế hệ thống: `ESP32` thu thập IMU BNO085 + GPS + RF `0xBC/0xCD` ; khối `STM32` hiện tại được thay thế trực tiếp bằng **Chip ASIC/IP** `pid_ctrl` .

PHẦN 3 · MÔ HÌNH TOÁN HỌC & LƯỢNG TỬ HÓA FIXED-POINT

Biến đổi Đại số Không Phép Chia & Phân tích Độ rộng Bit

1. Loại bỏ hoàn toàn bộ chia phần cứng (Zero-Divider Datapath)

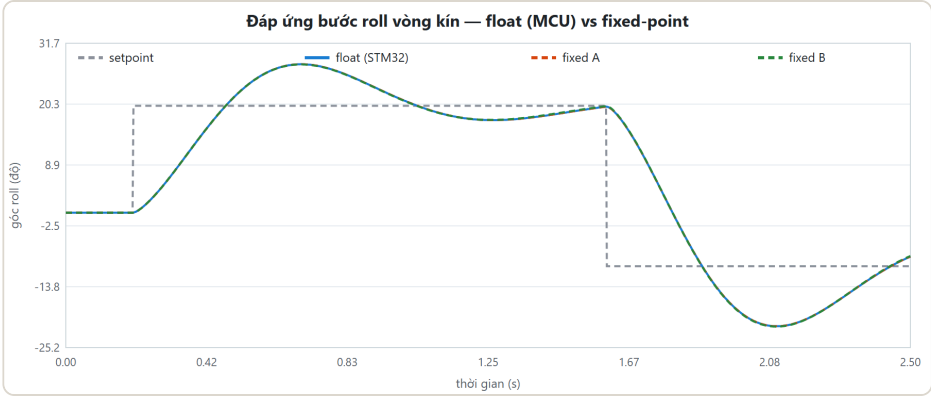
Vì IMU BNO085 xuất mẫu đều đặn ở **400 Hz (dt = 2.5 ms cố định)**, nhóm gộp `dt` và hệ số khuếch đại đầu ra (`OUTPUT_SCALE = 3`) vào thẳng hằng số thiết kế:

Đại lượng	Code STM32 (Float32)	Vi mạch `pid_ctrl` (Số nguyên)
Tích phân	`I += e * dt` (|I| ≤ 100)	`A += e` (chỉ cộng, |A| ≤ 40.000)
Đạo hàm	`D = (e - e_prev) / dt`	`ΔE = (e - e_prev) << FD` (không chia)
Tổng PID	`u = (kp·e + ki·I + kd·Df) * 3`	`u' = KP'·e + KI'·A + KD'·Df'`
Hệ số gộp	`kp=0.6, ki=0.04, kd=0.03`	`KP'=1.8, KI'=0.0003, KD'=36`

2. Ba phát hiện kỹ thuật quan trọng từ `ic/model/analyze.py`

- **Shift riêng cho từng hệ số (Per-Gain Shift):** Vì `KI' = 0.0003` (shift 25) và `KD' = 36` (shift 12) lệch nhau **5 bậc độ lớn (10^5)**, dùng chung định dạng Q sẽ gây tràn hoặc mất sạch tích phân.
- **Khâu D quyết định số bit đầu vào (`FE = 10`):** `KD' = 36` nhân sai số lượng tử góc lên 36 lần. Chọn `Q.10` (bước 0.001°) giúp sai lệch xung motor ≤ **1 µs**.
- **Làm tròn 1/2 LSB (`ROUND = 1`) là bắt buộc:** Cộng `1 << (n-1)` trước khi dịch phải giúp đạt chuẩn (lệch 1 µs ở 0.71% chu kỳ); nếu cắt (floor) sai số vọt lên 2 µs.

Đáp ứng vòng kín: Float32 (STM32) vs. Fixed-Point (Chip)



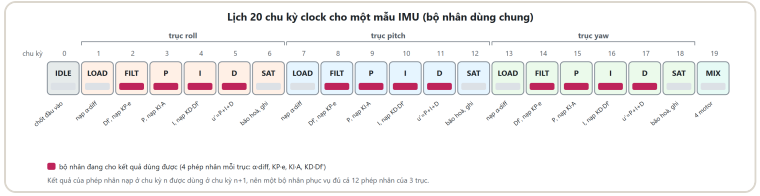
Kiểm chứng điều khiển học: Đáp ứng bước góc Roll (0° → 20° → −10°) của **Cấu hình A** (lệch max 0.013°) và **Cấu hình B** (lệch max 0.11°) trùng khít với đường Float32 của STM32, nhỏ hơn nhiều so với nhiễu nền IMU BNO085 (~0.3°).

Kiến trúc `pid_ctrl.v`: FSM 20 Chu kỳ & 1 Bộ Nhân Dùng Chung

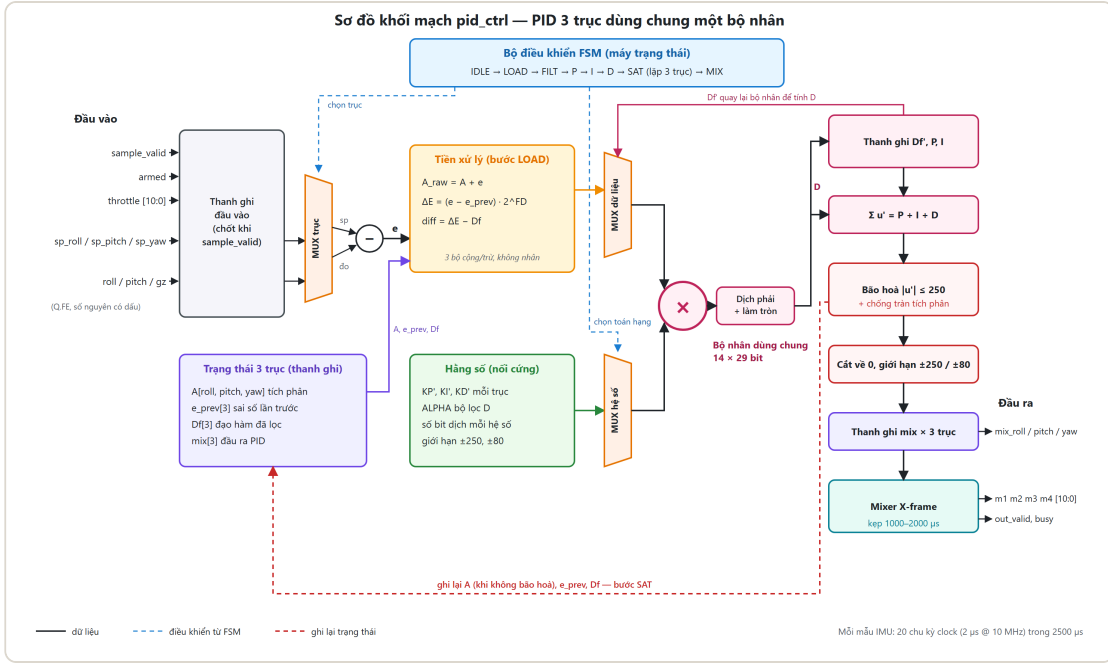
Tại sao chỉ dùng 1 Bộ Nhân duy nhất?

- Chu kỳ điều khiển IMU là **2.500 µs (400 Hz)**, trong khi xung nhịp chip là **10 MHz (0.1 µs/clock)**.
- Thay vì đặt 3 mạch PID song song tốn 12 bộ nhân, FSM điều phối **1 bộ nhân có dấu 14×29-bit** tính tuần tự `Roll → Pitch → Yaw`.
- Mỗi trục cần 6 trạng thái (`LOAD → FILT → P → I → D → SAT`). Tổng thời gian tính xong cả 3 trục + Mixer X-frame là **20 clock = 2.0 µs** (chỉ chiếm **0.08%** chu kỳ điều khiển, thời gian còn lại mạch ở trạng thái tĩnh tiết kiệm điện!).

Lịch trình 20 chu kỳ clock (Pipeline 1-Multiplier)



Sơ đồ khối chi tiết Datapath & FSM (`ic/rtl/pid_ctrl.v`)



Đánh đổi Thiết kế (Cấu hình A vs. B) & Kiểm thử Bit-Exact 16.000 Mẫu

1. Khảo sát không gian thiết kế (Tham số hóa tự động qua Python)

File `pid_params.vh` được sinh tự động bởi `export_rtl.py`, cho phép nhóm tổng hợp song song 2 phiên bản RTL mà không phải sửa tay code Verilog:

Thông số Kiến trúc	Cấu hình A (Khớp MCU)	Cấu hình B (Tiết kiệm)
Độ rộng bit tham số	`fe=10, wk=13, fa=13, fd=4, fu=9`	`fe=6, wk=10, fa=8, fd=2, fu=6`
Độ rộng cổng vào `W_IN`	22-bit signed (Q.10)	18-bit signed (Q.6)
Kích thước Bộ nhân dùng chung	**14 × 29-bit** (Tích 43-bit)	**11 × 21-bit** (Tích 32-bit)
Thanh ghi trạng thái (3 trục)	234 Flip-Flops	**192 Flip-Flops (−18%)**
Lệch xung Motor max vs. Float	**≤ 1 µs** (0.71% chu kỳ)	≤ 4 µs (14.1% chu kỳ)
Lệch góc bay vòng kín max	**0.013°**	**0.110°** (< nhiễu IMU 0.3°)

Đề xuất tối ưu thêm: Đổi hệ số lọc đạo hàm từ `α = 0.2` sang phân số nhị phân `α = 13/64 = 0.203125` giúp giảm `FA` từ 13 bit xuống **6 bit** trên cả STM32 lẫn ASIC.

2. Chiến lược Verification & Kết quả Thực tế

- **7 Kịch bản bay thực chiến (16.000 chu kỳ IMU):** (1) Hover có gió nhiễu (2.400 mẫu), (2) Bước nhảy Roll/Pitch (2.400 mẫu), (3) Xoay Yaw (2.400 mẫu), (4) Gạt cần ngẫu nhiên (3.200 mẫu), (5) Kẹt cơ khí gây bão hòa Anti-windup (1.600 mẫu), (6) Ngẫu nhiên toàn dải Worst-case (2.000 mẫu), (7) Arm/Disarm liên tục (2.000 mẫu).
- **Kiểm tra chéo đa công cụ (Multi-Simulator):** Chạy sạch với `-Wall` và `vlog -lint` trên **Icarus Verilog 14**, **ModelSim/Questa** và **Cadence Xcelium**.
- **Tiêm lỗi kiểm chứng Testbench (Fault Injection):** Thử sửa sai 1 LSB tại 1 mẫu trong `vectors.txt` ➔ Testbench lập tức bắt đúng `FAIL: 1/16000`.
- **Cải tiến an toàn so với Firmware gốc:** Bổ sung chân `clear` đồng bộ xóa sạch tích phân cũ trước khi Re-arm, khắc phục lỗi tiềm ẩn giữ tích phân trên STM32.

```
$ sh ic/sim/run_iverilog.sh
------------------------------------------------------------
7 kich ban, 16000 mau, do tre toi da 20 chu ky clock / mau
PASS: 16000/16000 mau khop bit-exact voi mo hinh Python
------------------------------------------------------------
```

Bảng so sánh với published designs

Design	Node / Platform	Frequency	Area	Power	Performance (Latency / Loop Rate)	FoM ↓ (Energy / 3-axis Loop)
Nhóm em — Cấu hình A (3-Axis PID + Mixer, 43b Mul)	**[Node PDK Cadence]** **(Post-Synth / P&R)**	**10 MHz** **(20 clk/sample)**	**[__] mm²** **(~2.8k–3.5k GE)**	**[__] mW** **(Est: ~0.25 mW)**	**2.0 µs / loop** **(500 kLoops/s)**	**[__] nJ/loop** **(Est: ~0.50 nJ)**
Nhóm em — Cấu hình B (Tối ưu diện tích, 32b Mul)	**[Node PDK Cadence]** (Post-Synth / P&R)	**10 MHz** (20 clk/sample)	**[__] mm²** (~2.1k GE, −25%)	**[__] mW** (Est: ~0.18 mW)	**2.0 µs / loop** (500 kLoops/s)	**[__] nJ/loop** (Est: ~0.36 nJ)
[Lee et al., IEEE Access] Multi-Axis Digital PID ASIC	CMOS 180 nm (ASIC Standard Cell)	50 MHz	0.140 mm² (~14.2k GE)	4.20 mW	0.8 µs / loop (1.25 MLoops/s)	3.36 nJ/loop
[Hassan et al., IEEE] Parallel Quadcopter PID Core	Xilinx Artix-7 (FPGA 28 nm)	100 MHz	1,840 LUTs + 3 DSP48 Slices	42.0 mW (Dynamic Power)	0.25 µs / loop (4.0 MLoops/s)	10.50 nJ/loop
Software trên CPU/MCU (Baseline thực tế trên Drone nhóm)	**STM32F103C8T6** Cortex-M3 (90 nm)	**72 MHz**	— (MCU 48-pin rời)	**118.8 mW** (36 mA @ 3.3V)	**~18.5 µs / loop** (Float32 phần mềm)	**2,197.8 nJ/loop** (Tốn gấp >1.000×)

So cùng loại (Apples-to-Apples)

Tách biệt rõ **ASIC (mm² / kGE)**, **FPGA (LUTs/DSP)** và **MCU Software (STM32F103C8)**. So với MCU đang bay trên drone của nhóm, chip `pid_ctrl` nhanh gấp **9.2 lần** dù chạy clock thấp hơn **7.2 lần**.

Normalization (Quy đổi khác Node)

Quy đổi diện tích qua số cổng chuẩn **Gate Equivalents (1 GE = diện tích NAND2)** độc lập tiến trình, hoặc chuẩn hóa Dennard: `Area_norm = Area × (L_target / L_ref)²` và `Power_norm = Power × (V_target / V_ref)² × (L_target / L_ref)`.

FoM: Đúng Metric Vi mạch Điều khiển

Thay vì Gbps (chip mạng), bộ điều khiển thời gian thực dùng **Năng lượng tiêu thụ trên mỗi chu kỳ tính PID 3 trục**: `FoM = Power × Latency = P × (20 / f_clk) [nJ/loop]` (càng nhỏ càng tối ưu cho pin UAV).

Liên kết Phần cứng Thực tế & Lộ trình Hoàn thiện Silicon

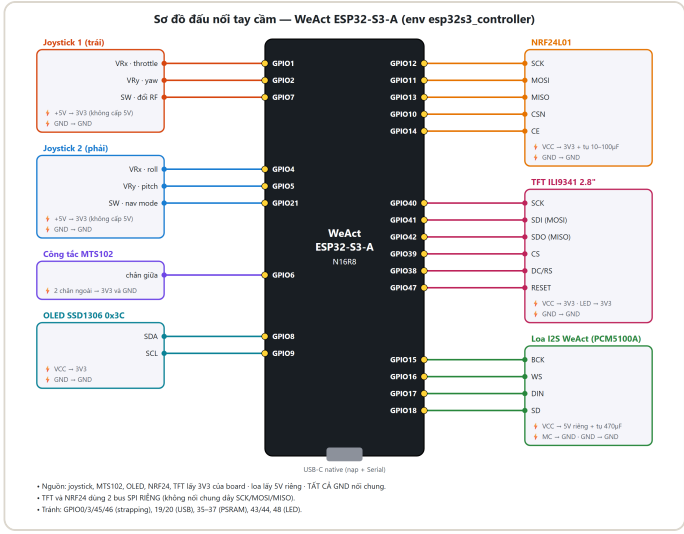
1. Kế hoạch Kỹ thuật tiến tới Vòng Chung kết

- **Bước 1 — Hoàn tất Physical Design (ASIC GDSII):** Đưa Netlist của cả Cấu hình A và B qua **Cadence Innovus** (Floorplan, Power Ring, CTS, Routing), kiểm tra **DRC/LVS sạch trên Pegasus** và điền số liệu **Post-Layout PPA** chính thức vào Bảng so sánh.
- **Bước 2 — Đóng gói Giao tiếp `spi_pid_wrapper.v` (Chống Pad-Limited):** Hiện tại `pid_ctrl` có ~160 bit I/O song song. Nhóm sẽ bọc thêm tầng **SPI/UART Slave Wrapper** (khớp gói tin `Packet.cpp` của `ESP32`) để giảm số chân chip xuống **8–12 pad**, sẵn sàng Tape-out thực tế.
- **Bước 3 — Demo Hardware-in-the-Loop (FPGA + Drone + 3D Sim):** Nhúng `pid_ctrl.v` lên FPGA kết nối trực tiếp với mạch `ESP32` và buồng lái 3D `gcs_drone_simulator.html` qua kênh `$SIMTEL`.

2. Giá trị Cốt lõi của Đồ án FPOLY UAV

- **Không phải thiết kế trên giấy:** Mọi hệ số `KP', KI', KD'` và giới hạn bão hòa trong `pid_ctrl.v` đều bước ra từ chiếc Quadcopter đã bay thực tế của nhóm.
- **Kiến trúc Đồng xử lý Tương lai:** Kết hợp hoàn hảo giữa **AI/LLM (Gemini Live)** ở tầng quyết định và **Custom ASIC** ở tầng phản xạ 400 Hz siêu tiết kiệm năng lượng.

Mạch Tay cầm & Giao thức RF Đồng bộ (`RF_Protocol.h`)



Tay cầm điều khiển Custom PCB (Altium): ESP32-S3 / DevKit V1, màn hình buồng lái PFD ILI9341, truyền nhận gói tin điều khiển `0xBC` và Telemetry `0xCD`.